# Operaciones de Machine Learning

Proyecto 2
Nivel 3

Pontificia Universidad Javeriana
Cristian Javier Diaz Alvarez

24 de abril de 2026

## 1. Descripción

Este proyecto tiene como objetivo diseñar, implementar y desplegar una arquitectura integral de MLOps sobre Kubernetes. El sistema debe cubrir el ciclo completo de vida de un modelo de Machine Learning: ingesta de datos por lotes, almacenamiento de información cruda, procesamiento y limpieza, entrenamiento periódico, registro de experimentos, versionamiento de modelos, selección automática del mejor modelo, despliegue de inferencia mediante una API, consumo desde una interfaz gráfica, pruebas de carga y observabilidad del servicio.

El caso de uso se basa en un conjunto de datos clínicos de pacientes con diabetes. El estudiante deberá construir una solución que simule un escenario real de producción, en el cual los datos llegan de forma incremental, son almacenados inicialmente sin transformación, posteriormente son procesados para entrenamiento y finalmente son utilizados para construir modelos versionados en MLflow.

La arquitectura debe estar desplegada completamente en un clúster local de Kubernetes. Todos los componentes del sistema deben ejecutarse como contenedores, y las imágenes construidas por el estudiante deben publicarse en DockerHub o en un registro de contenedores equivalente, de forma que puedan ser utilizadas directamente por los manifiestos de Kubernetes.

El sistema debe implementar, como mínimo, los siguientes componentes:

- Un DAG de Airflow encargado de orquestar el flujo de ingesta, procesamiento, entrenamiento y registro del modelo.
- Una base de datos relacional para almacenar los datos crudos (*raw data*).
- Una base de datos relacional o conjunto de tablas independientes para almacenar los datos procesados (*clean data*).
- Un servidor de MLflow con almacenamiento externo de metadatos y artefactos.
- Un sistema de almacenamiento de objetos, como MinIO, para almacenar los artefactos generados por MLflow.
- Una API desarrollada con FastAPI para realizar inferencia usando el modelo productivo registrado en MLflow.
- Una interfaz gráfica desarrollada con Streamlit para consumir la API de inferencia.
- Una instancia de Locust para ejecutar pruebas de carga sobre la API.
- Un stack de observabilidad compuesto, como mínimo, por Prometheus y Grafana.

La ejecución del DAG debe simular un proceso de carga incremental. El conjunto de entrenamiento no debe ser cargado en una sola operación, sino en lotes de máximo 15.000 registros por ejecución o por tarea de carga. En cada ejecución del flujo, Airflow debe cargar nuevos datos en la tabla *raw*, procesarlos, almacenarlos en una tabla *clean*, entrenar al menos un modelo y registrar el experimento en MLflow.

El modelo con mejor desempeño, de acuerdo con una métrica definida y justificada por el estudiante, debe ser registrado como modelo productivo. La API de inferencia no puede cargar un archivo local ni depender de una versión fija escrita en el código. Debe consultar dinámicamente el modelo productivo desde MLflow, ya sea usando el stage `Production` o un alias como `champion`, según la versión de MLflow utilizada.

Adicionalmente, cada inferencia realizada desde la API debe quedar registrada en la base de datos en una tabla de datos crudos o en una tabla específica de eventos de inferencia. Este registro debe permitir que las predicciones realizadas puedan ser utilizadas posteriormente como insumo para futuros procesos de entrenamiento, auditoría o monitoreo.

La observabilidad debe permitir evidenciar el comportamiento de la API durante las pruebas de carga. Como mínimo, se deben exponer métricas en el endpoint `/metrics`, recolectarlas con Prometheus y visualizarlas en Grafana. Durante la sustentación se debe mostrar el efecto de una prueba de carga ejecutada con Locust sobre los dashboards de Grafana.

# 2. Objetivos del proyecto

## 2.1. Objetivo general

Implementar una arquitectura de MLOps desplegada en Kubernetes que permita automatizar el ciclo de vida de un modelo de Machine Learning, desde la ingesta incremental de datos hasta la inferencia productiva, incorporando versionamiento de modelos, pruebas de carga y observabilidad.

## 2.2. Objetivos específicos

1. Diseñar un flujo orquestado con Airflow para cargar datos por lotes, procesarlos y entrenar modelos de Machine Learning.
2. Separar adecuadamente las capas de almacenamiento de datos crudos, datos procesados y registros de inferencia.
3. Registrar experimentos, métricas, parámetros, artefactos y modelos usando MLflow.
4. Seleccionar automáticamente el mejor modelo entrenado y marcarlo como modelo productivo.
5. Desplegar una API de inferencia que consuma dinámicamente el modelo productivo registrado en MLflow.
6. Construir una interfaz gráfica simple que permita enviar datos a la API y visualizar la predicción generada.
7. Registrar cada solicitud de inferencia para habilitar futuros procesos de reentrenamiento.
8. Ejecutar pruebas de carga sobre la API usando Locust.
9. Implementar observabilidad básica de la API usando Prometheus y Grafana.
10. Desplegar todos los componentes en Kubernetes, definiendo recursos mínimos y máximos para cada contenedor.

# 3. Descripción del dataset

El conjunto de datos representa 10 años (1999-2008) de atención clínica en 130 hospitales de EE.UU. y redes de entrega integradas. Incluye más de 50 características que representan los resultados del paciente y del hospital. Se extrajo información de la base de datos para encuentros que cumplían con los siguientes criterios.

1. Es un encuentro de paciente hospitalizado (un ingreso en el hospital).
2. Es un encuentro diabético, es decir, durante el cual se ingresó al sistema como diagnóstico cualquier tipo de diabetes.
3. La duración de la estadía fue de al menos 1 día y como máximo 14 días.
4. Se realizaron pruebas de laboratorio durante el encuentro.
5. Se administraron medicamentos durante el encuentro.

Los datos contienen atributos tales como número de paciente, raza, sexo, edad, tipo de ingreso, tiempo en el hospital, especialidad médica del médico que lo admitió, número de pruebas de laboratorio realizadas, resultado de la prueba de HbA1c, diagnóstico, número de medicamentos, medicamentos para la diabetes, número de pacientes ambulatorios. , hospitalización y visitas de emergencia en el año anterior a la hospitalización, etc. Diabetes 130-US hospitals for years 1999-2008

## 3.1. Cargar el Dataset

Utilice las siguientes celdas para cargar el conjunto de datos **modificado** en su espacio de trabajo.

```
## download the dataset
# Directory of the raw data files
_data_root = './data/Diabetes'

# Path to the raw training data
_data_filepath = os.path.join(_data_root, 'Diabetes.csv')

# Download data
os.makedirs(_data_root, exist_ok=True)
if not os.path.isfile(_data_filepath):
    #https://archive.ics.uci.edu/ml/machine-learning-databases/covtype/
    url = 'https://docs.google.com/uc?export= \
    download&confirm={{VALUE}}&id=1k5-1caezQ3zWJbKaiMULTGq-3sz6uThC'
    r = requests.get(url, allow_redirects=True, stream=True)
    open(_data_filepath, 'wb').write(r.content)
```

Este es todo el conjunto de datos, debe realizarse completo su procesamiento, desde análisis, limpieza, ingeniería de características y creación de subconjuntos de entrenamiento, validación y test. Este procesamiento se tendrá en cuenta para la asignación de la nota. Si desea descargar el archivo de manera directa puede encontrarlo Acá

# 4. Requerimientos técnicos obligatorios

## 4.1. Despliegue en Kubernetes

Todos los componentes del sistema deben desplegarse en un clúster local de Kubernetes. No se aceptará como entrega final una arquitectura ejecutada únicamente con `docker-compose`. Docker Compose puede utilizarse durante el desarrollo, pero la sustentación y la entrega deben demostrar el despliegue funcional en Kubernetes.

Cada componente debe contar, como mínimo, con:

- `Deployment`, `StatefulSet` o recurso equivalente, según corresponda.

- `Service` para comunicación interna o exposición controlada.
- `ConfigMap` para configuración no sensible.
- `Secret` para credenciales, contraseñas, tokens o variables sensibles.
- `PersistentVolumeClaim` cuando el componente requiera persistencia.
- Definición explícita de `resources.requests` y `resources.limits`.
- Probes de salud cuando aplique: `readinessProbe` y `livenessProbe`.

Las imágenes propias del proyecto deben construirse y publicarse en DockerHub o en un registro de contenedores equivalente. Los manifiestos de Kubernetes no deben depender de imágenes locales que solo existan en la máquina del estudiante.

## 4.2. Orquestación con Airflow

Airflow debe ejecutarse dentro del clúster de Kubernetes. El estudiante puede apoyarse en la guía disponible en el repositorio del curso para desplegar Airflow y cargar DAGs personalizados:

Guía de despliegue de Airflow en Kubernetes

El DAG principal debe incluir, como mínimo, las siguientes tareas:

1. Validación de disponibilidad del archivo fuente.
2. Carga incremental del archivo en lotes hacia la tabla de datos crudos.
3. Validación básica de calidad de datos.
4. Preprocesamiento y transformación de datos.
5. Almacenamiento de datos procesados en una tabla independiente.
6. Separación de datos en entrenamiento, validación y prueba.
7. Entrenamiento de uno o varios modelos.
8. Registro de parámetros, métricas, artefactos y modelo en MLflow.
9. Comparación de desempeño contra modelos anteriores.
10. Promoción automática del mejor modelo a producción o asignación del alias productivo.

El DAG debe estar diseñado para ser reejecutable. Si una tarea falla, el error debe ser visible en Airflow y no debe dejar el sistema en un estado inconsistente. Se valorará positivamente el uso de tareas idempotentes, control de duplicados y validaciones antes de insertar datos.

## 4.3. Almacenamiento de datos

El sistema debe separar explícitamente las siguientes capas de datos:

- **RAW DATA:** datos cargados desde el archivo fuente sin transformaciones destructivas.
- **CLEAN DATA:** datos limpios, transformados y listos para entrenamiento.
- **INFERENCE LOGS:** registros generados por la API cada vez que se realiza una predicción.

Estas capas pueden implementarse como bases de datos independientes o como esquemas/tablas separadas dentro de una misma instancia de PostgreSQL. En cualquier caso, la separación debe ser clara y justificarse técnicamente.

La tabla de datos crudos debe permitir identificar, como mínimo:

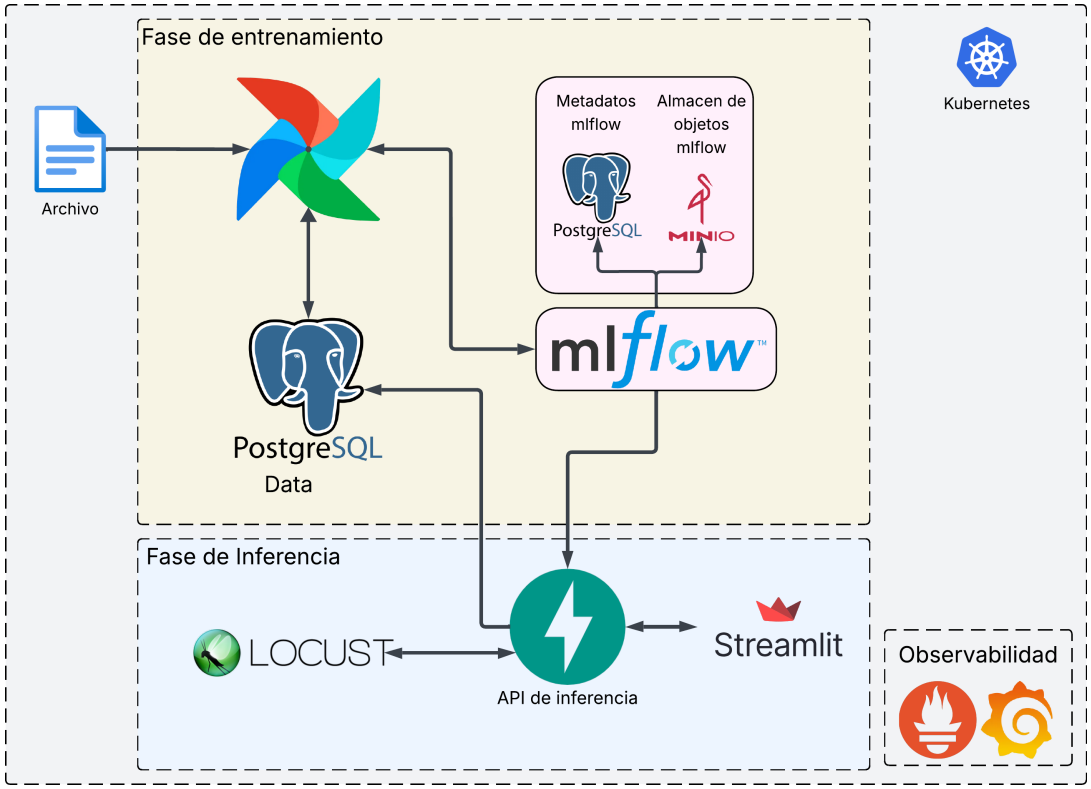
- lote de carga;
- fecha y hora de carga;
- origen del dato;
- estado del registro;
- identificador único del registro, si aplica.

La tabla de inferencias debe almacenar, como mínimo:

- fecha y hora de la inferencia;
- datos de entrada recibidos por la API;
- predicción generada;
- probabilidad o score, si el modelo lo permite;
- nombre del modelo utilizado;
- versión del modelo o alias productivo;
- tiempo de respuesta de la inferencia;
- identificador de la solicitud.

## 5. Arquitectura

La arquitectura propuesta representa un flujo típico de MLOps en el cual los datos, los modelos y los servicios de inferencia se gestionan como componentes independientes pero integrados. El sistema está dividido en dos fases principales: fase de entrenamiento y fase de inferencia.



## 5.1. Fase de entrenamiento

La fase de entrenamiento inicia con la carga incremental del archivo fuente. Airflow debe orquestar la lectura del archivo, dividir los datos en lotes y almacenar cada lote en la capa de datos crudos. Esta capa representa el estado original de la información y debe conservarse como fuente histórica para auditoría y reprocesamiento.

Posteriormente, Airflow debe ejecutar las tareas de limpieza, transformación e ingeniería de características. El resultado de este proceso debe almacenarse en la capa de datos procesados. Esta separación es obligatoria, ya que permite diferenciar entre los datos recibidos originalmente y los datos que han sido preparados para entrenamiento.

A partir de los datos procesados, el DAG debe entrenar uno o varios modelos de Machine Learning. Cada ejecución de entrenamiento debe registrar en MLflow los parámetros utilizados, las métricas obtenidas, los artefactos relevantes y el modelo resultante. El estudiante debe justificar la métrica principal usada para seleccionar el mejor modelo. Por ejemplo, en un problema de clasificación clínica, no basta con reportar *accuracy*; se debe analizar si métricas como *recall*, *precision*, *F1-score*, *ROC-AUC* o métricas por clase son más adecuadas según el objetivo del problema.

El mejor modelo debe quedar disponible en el registro de modelos de MLflow como modelo productivo. La promoción del modelo debe ser automática o, como mínimo, reproducible mediante una tarea definida en el DAG. No se aceptará como solución cambiar manualmente el nombre de un archivo, copiar un modelo local o modificar el código de la API para apuntar a una versión específica.

## 5.2. Fase de inferencia

La fase de inferencia está compuesta por una API desarrollada con FastAPI y una interfaz gráfica desarrollada con Streamlit. La interfaz debe permitir al usuario ingresar valores, enviar la solicitud a la API y visualizar la predicción generada.

La API debe cargar el modelo productivo desde MLflow. Esto implica que, si el modelo productivo cambia en MLflow, la API debe poder usar la nueva versión sin requerir cambios en el código fuente. Se recomienda implementar un mecanismo de carga controlada del modelo, evitando descargarlo en cada solicitud si esto afecta el rendimiento. El estudiante debe explicar la estrategia utilizada: carga al iniciar la API, recarga bajo demanda, caché temporal o consulta por versión.

Cada solicitud de inferencia debe registrarse en la base de datos. Este registro es obligatorio porque simula un escenario real en el cual los datos generados en producción pueden convertirse en insumo para futuros entrenamientos, monitoreo de deriva, auditoría o análisis de desempeño.

## 5.3. Observabilidad y pruebas de carga

La API debe exponer métricas en el endpoint `/metrics`. Prometheus debe recolectar estas métricas periódicamente y Grafana debe visualizarlas en un dashboard.

Como mínimo, el dashboard debe incluir:

- número total de solicitudes;
- solicitudes por segundo;
- latencia promedio;
- latencia por percentiles, por ejemplo p50, p95 o p99;
- número de errores;
- tasa de error;

- uso de CPU y memoria del pod de la API, si está disponible.

Locust debe utilizarse para simular usuarios concurrentes consumiendo la API. Durante la sustentación se debe evidenciar cómo cambian las métricas en Grafana mientras se ejecuta la prueba de carga. No es suficiente mostrar que Locust se ejecuta; se debe explicar qué comportamiento se observa en la API y cuál parece ser su límite operativo bajo las condiciones de recursos configuradas.

# 6. Componentes

## 6.1. Airflow

Airflow es el componente responsable de orquestar el flujo de datos y entrenamiento. Debe ejecutarse dentro del clúster de Kubernetes y contener al menos un DAG principal del proyecto.

El DAG debe encargarse de:

- cargar datos por lotes desde el archivo fuente;
- insertar los registros en la tabla de datos crudos;
- ejecutar validaciones mínimas de calidad;
- transformar los datos y generar la tabla de datos procesados;
- separar los datos en entrenamiento, validación y prueba;
- entrenar modelos;
- registrar experimentos en MLflow;
- seleccionar el mejor modelo;
- registrar o promover el modelo productivo.

El estudiante debe explicar la periodicidad del DAG, las dependencias entre tareas y qué ocurre si una tarea falla. También debe indicar cómo evita duplicar registros cuando el DAG se ejecuta más de una vez.

## 6.2. Base de datos de datos crudos

La base de datos de datos crudos almacena la información tal como llega al sistema. Esta capa no debe sobrescribirse en cada ejecución. El cargue debe realizarse por lotes de máximo 15.000 registros, simulando un proceso incremental.

La tabla de datos crudos debe conservar información suficiente para identificar qué registros pertenecen a cada lote. Se recomienda incluir campos como `batch_id`, `load_timestamp`, `source_file`, `row_hash` y `status`.

## 6.3. Base de datos de datos procesados

La base de datos o tabla de datos procesados almacena la información después de aplicar limpieza, transformación e ingeniería de características. Esta tabla debe ser la fuente utilizada para el entrenamiento de modelos.

El estudiante debe documentar las transformaciones realizadas, justificar las variables eliminadas o modificadas, explicar el tratamiento de valores nulos y describir la estrategia de codificación de variables categóricas.

## 6.4. MLflow

MLflow debe utilizarse para el seguimiento de experimentos y el registro de modelos. El servidor de MLflow debe estar desplegado en Kubernetes y debe utilizar almacenamiento externo para metadatos y artefactos.

No se permite usar SQLite como backend de metadatos en la entrega final. Se recomienda usar PostgreSQL como backend store y MinIO como artifact store.

Cada experimento debe registrar, como mínimo:

- identificador de la ejecución;
- fecha y hora de ejecución;
- parámetros del modelo;
- métricas de entrenamiento y validación;
- métrica principal de selección;
- artefactos relevantes;
- modelo serializado;
- versión registrada del modelo.

El mejor modelo debe quedar registrado como modelo productivo. Dependiendo de la versión de MLflow utilizada, esto puede implementarse mediante el stage `Production` o mediante un alias como `champion`.

## 6.5. MinIO o sistema de almacenamiento de objetos

El sistema de almacenamiento de objetos debe almacenar los artefactos generados por MLflow. Puede utilizarse MinIO desplegado en Kubernetes o un servicio externo compatible con S3.

El estudiante debe configurar correctamente las credenciales de acceso, el bucket de artefactos y las variables necesarias para que MLflow pueda guardar y recuperar modelos, métricas, gráficos u otros archivos generados durante la experimentación.

## 6.6. API de inferencia

La API debe desarrollarse con FastAPI y debe exponer, como mínimo, los siguientes endpoints:

- `/health`: permite verificar que la API está activa.
- `/predict`: recibe los datos de entrada y retorna la predicción.
- `/model-info`: retorna el nombre, versión y estado del modelo actualmente usado.
- `/metrics`: expone métricas compatibles con Prometheus.

La API debe consultar el modelo productivo desde MLflow y no debe depender de un archivo local fijo. Además, debe registrar cada inferencia en la base de datos.

La respuesta del endpoint `/predict` debe incluir, como mínimo:

- predicción generada;
- score o probabilidad, si aplica;
- modelo utilizado;
- versión o alias del modelo;
- tiempo de procesamiento.

### 6.7. Interfaz gráfica

La interfaz gráfica debe desarrollarse con Streamlit o una herramienta equivalente. Su objetivo no es evaluar diseño web avanzado, sino permitir la interacción clara con el modelo.

La interfaz debe permitir:

- ingresar valores para realizar inferencia;
- cargar valores de ejemplo;
- enviar la solicitud a la API;
- visualizar la predicción;
- visualizar la versión del modelo utilizada;
- mostrar errores de validación cuando los datos ingresados no sean válidos.

La interfaz no debe comunicarse directamente con MLflow ni con la base de datos para realizar inferencias. La inferencia debe hacerse exclusivamente a través de la API.

### 6.8. Locust

Locust debe desplegarse como un componente del sistema o ejecutarse de forma controlada contra la API desplegada en Kubernetes. El estudiante debe definir un escenario de prueba que simule usuarios realizando solicitudes al endpoint `/predict`.

La prueba debe reportar, como mínimo:

- número de usuarios simulados;
- tasa de llegada o *spawn rate*;
- número total de solicitudes;
- solicitudes exitosas;
- solicitudes fallidas;
- latencia promedio;
- percentiles de latencia;
- punto aproximado en el cual la API empieza a degradarse.

### 6.9. Prometheus y Grafana

Prometheus debe recolectar métricas de la API desde el endpoint `/metrics`. Grafana debe permitir visualizar el comportamiento del servicio durante la ejecución normal y durante las pruebas de carga.

El estudiante debe entregar al menos un dashboard funcional. Se valorará positivamente que el dashboard incluya métricas de aplicación y métricas de infraestructura.

## 7. Entregables

La entrega del proyecto debe incluir:

1. Repositorio público con el código fuente del proyecto.
2. Dockerfiles de los componentes desarrollados por el estudiante.
3. Evidencia de publicación de imágenes en DockerHub o registro equivalente.

4. Manifiestos de Kubernetes o Helm Charts necesarios para desplegar la solución.
5. DAGs de Airflow.
6. Scripts de creación de tablas o migraciones de base de datos.
7. Código de entrenamiento y registro en MLflow.
8. Código de la API de inferencia.
9. Código de la interfaz gráfica.
10. Archivo de prueba de Locust.
11. Configuración de Prometheus.
12. Dashboard de Grafana exportado en formato JSON o documentado en el repositorio.
13. Archivo README con instrucciones claras de despliegue, ejecución y validación.
14. Video de sustentación publicado en YouTube con duración máxima de 10 minutos.

# 8. Criterios de evaluación

La calificación del proyecto se distribuirá de la siguiente manera:

- **Despliegue en Kubernetes e integración general de la arquitectura (20 %):** todos los componentes deben ejecutarse en Kubernetes, con servicios, persistencia, variables de entorno, secretos y límites de recursos correctamente definidos.
- **Orquestación con Airflow y procesamiento de datos (20 %):** el DAG debe cargar datos por lotes, almacenar datos crudos, procesar información, generar datos limpios y ejecutar el entrenamiento de forma reproducible.
- **Experimentación, versionamiento y selección de modelos con MLflow (20 %):** se deben registrar experimentos completos, comparar modelos y publicar el mejor modelo como productivo sin depender de cambios manuales en el código.
- **API de inferencia e interfaz gráfica (15 %):** la API debe consumir dinámicamente el modelo productivo desde MLflow, registrar las inferencias y ser consumida desde una interfaz gráfica funcional.
- **Observabilidad y pruebas de carga (15 %):** la API debe exponer métricas, Prometheus debe recolectarlas, Grafana debe visualizarlas y Locust debe evidenciar el comportamiento del sistema bajo carga.
- **Sustentación y documentación (10 %):** el video debe explicar la arquitectura, decisiones técnicas, ejecución del DAG, experimentación, inferencia, observabilidad, pruebas de carga y principales dificultades encontradas.

# 9. Restricciones y condiciones de aceptación

Para que el proyecto sea considerado funcional, debe cumplir las siguientes condiciones mínimas:

1. El sistema debe desplegarse en Kubernetes. No se acepta una entrega final ejecutada únicamente en notebooks o Docker Compose.
2. El DAG de Airflow debe ejecutarse correctamente y evidenciar varias tareas independientes.
3. Los datos crudos y procesados deben almacenarse en tablas separadas.
4. El cargue de datos debe realizarse por lotes y no mediante una única inserción completa del dataset de entrenamiento.

5. MLflow debe contar con backend store y artifact store externos. No se acepta SQLite como backend final.
6. El modelo usado por la API debe provenir de MLflow.
7. La API no debe tener quemada una ruta local a un archivo de modelo.
8. Cada inferencia debe quedar registrada en la base de datos.
9. La API debe exponer métricas en `/metrics`.
10. Prometheus debe recolectar métricas de la API.
11. Grafana debe mostrar visualmente el efecto de una prueba de carga ejecutada con Locust.
12. Cada contenedor debe definir recursos mínimos y máximos mediante `requests` y `limits`.
13. Las imágenes propias deben estar disponibles desde un registro de contenedores.